

1] KthSmallestElement.java

```
import java.util.*;  
  
public class KthSmallestElement {  
    public static int findKthSmallest(List<Integer> list, int k)  
    {  
        Collections.sort(list); // Sort the list  
        return list.get(k-1); // Return k-th smallest  
    }  
  
    public static void main (String [] args) {  
        List<Integer> numbers = Arrays.asList(5, 3, 8, 1, 2);  
        int k = 3;  
        System.out.println ("Kth smallest: " + findKthSmallest  
                               (numbers, k));  
    }  
}
```

2 WordFrequencyMap.java

```
import java.util.*;

public class WordFrequencyMap {
    public static void main(String[] args) {
        String text = "apple banana apple orange banana apple";
        TreeMap<String, Integer> freqMap = new TreeMap<>();
        for (String word : text.split(" ")) {
            freqMap.put(word, freqMap.getOrDefault(word, 0) + 1);
        }
        System.out.println(freqMap); // output word frequencies
    }
}
```


3 QueueStackUsingPriorityQueue.java

```
import java.util.*;
```

```
public class QueueStackUsingPriorityQueue {
```

```
    static class PQQueue {
```

```
    } private PriorityQueue<int[]> queue = new PriorityQueue
```

```
        <> (Comparator.comparingInt(a -> a[1]));
```

```
    private int time = 0;
```

```
    public void offer (int val) {
```

```
        queue.offer (new int[] {val, time++});
```

```
    }
```

```
    public int poll () {
```

```
        return queue.poll () [0];
```

```
    }
```

```
    static class PQStack {
```

```
    private PriorityQueue<int[]> stack = new
```

```
        PriorityQueue<> ((a, b) -> b[1] - a[1]);
```

```
    private int time = 0;
```

```
    public void push (int val) {
```

```
        stack.offer (new int[] {val, time++});
```

```
    }
```



```
public int pop () {
```

```
    return stack.poll () [0];
```

```
} }
```

```
public static void main (String [] args) {
```

```
    PQQueue q = new PQQueue();
```

```
    q.offer (1); q.offer (2); q.offer (3);
```

```
    System.out.println ("Queue poll: " + q.poll ());
```

```
    PQStack s = new PQStack ();
```

```
    s.push (1); s.push (2); s.push (3);
```

```
    System.out.println ("Stack pop: " + s.pop ());
```

```
}
```

```
}
```

```
private PriorityQueue < int [] > stack =
```

```
new PriorityQueue < > ();
```

```
private int time = 0;
```

```
public void push (int val) {
```

```
    stack.offer (new int [] {val, time++});
```


[4] StudentTreeMap.java

```
import java.util.*;
```

```
class Student {
```

```
    String name;
```

```
    int age;
```

```
    public Student (String name, int age) {
```

```
        this.name = name; this.age = age;
```

```
    }
```

```
    public String toString() {
```

```
        return name + " (" + age + " )";
```

```
    }
```

```
public class StudentTreeMap {
```

```
    public static void main (String[] args) {
```

```
        TreeMap <Integer, Student> students = new TreeMap
```

```
        <> ();
```

```
        students.put (101, new Student ("Alice", 20));
```

```
        students.put (102, new Student ("Bob", 22));
```



```

for (Map.Entry <Integer, Student> entry :
     students.entrySet()) {
    System.out.println (entry.getKey() + " => " +
                        entry.getValue());
}
}
}

```

5 Linked List Equality.java

```

import java.util.*;

public class Linked List Equality {
    public static void main (String [] args) {
        LinkedList <Integer> list1 = new LinkedList <>
            (Arrays.asList (1, 2, 3));
        LinkedList <Integer> list2 = new LinkedList <>
            (Arrays.asList (1, 2, 3));
        System.out.println ("List equal ? " + list1.equals(list2));
    }
}

```


[5] EmployeeDepartmentMap.java

```
import java.util.*;
```

```
public class EmployeeDepartmentMap {
```

```
    public static void main (String [] args) {
```

```
        HashMap <Integer, String> employeeMap = new  
            HashMap <> ();
```

```
        employeeMap.put (1001, "HR");
```

```
        employeeMap.put (1002, "IT");
```

```
        employeeMap.put (1003, "Finance");
```

```
        for (Map.Entry <Integer, String> entry : employeeMap.  
            entrySet ()) {
```

```
            System.out.println ("ID: " + entry.getKey () + " => Dept:
```

```
                + entry.getValue ());
```

```
        }
```

```
    }
```

```
}
```